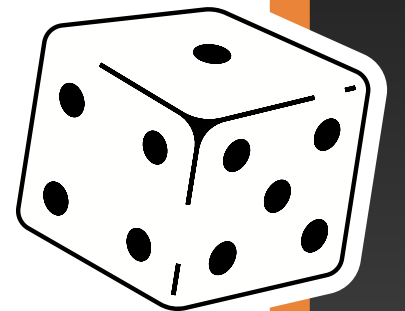
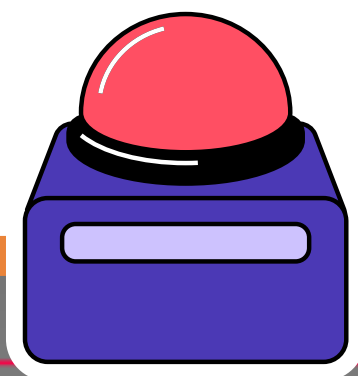




# INTRODUCTION TO JAVA

ALGORITHMS & OBJECT-ORIENTED PROGRAMMING  
WEEK 9  
JOSEPH ANDREAS

FOR INTERNAL USE ONLY



# AGENDA

1. JAVA OVERVIEW & INSTALLATION

2. BASIC STRUCTURE

3. INPUT AND OUTPUT

4. DATA TYPES

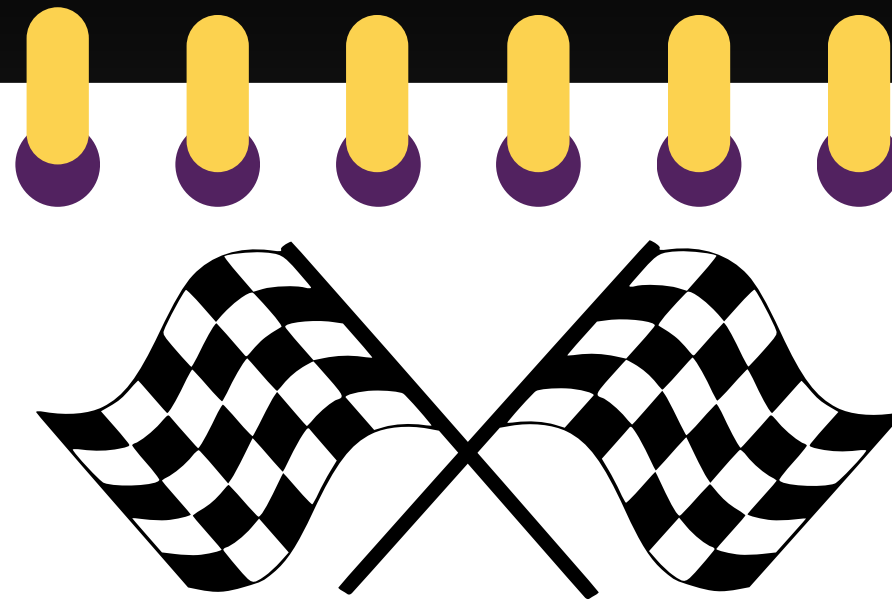
5. CONDITIONALS AND LOOPS

6. METHODS / FUNCTIONS ?

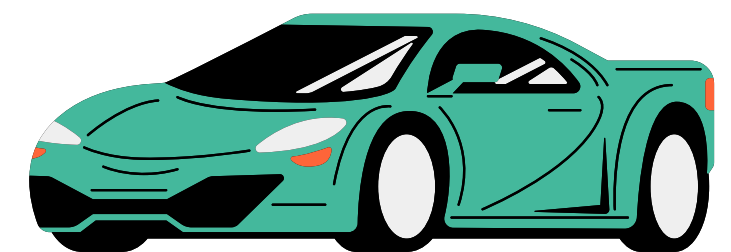
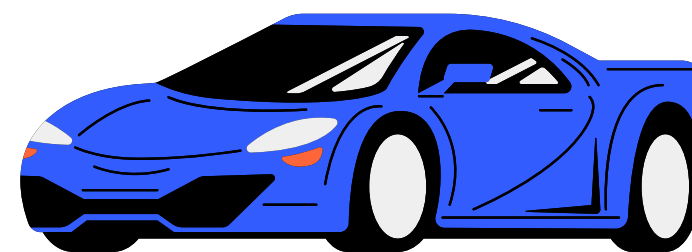
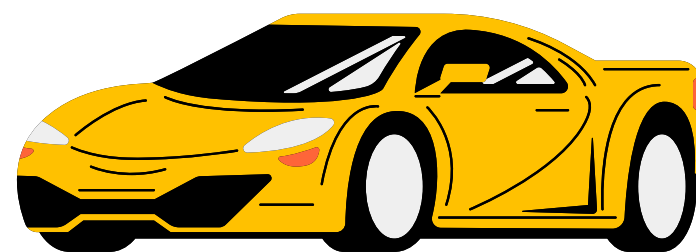
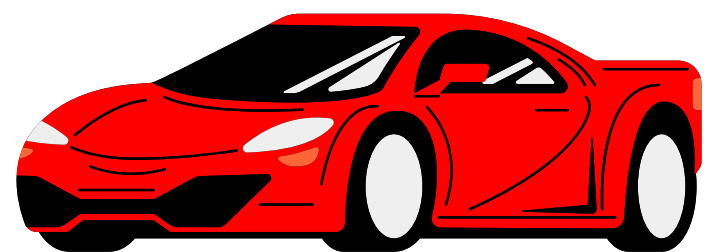


# START





# JAVA OVERVIEW & INSTALLATION



# INTRODUCTION TO JAVA

- High-level, object-oriented programming language
  - Object-oriented programming: Computer programming model that organizes software design around data, or objects, rather than functions and logic -> Gives a clear structure to programs and allows code to be reused, lowering development costs
- Works on different platforms (Windows, Mac, Linux, Raspberry Pi, etc.)
- One of the most popular programming languages in the world
- Close to C++ and C# -> Easy to switch to Java or vice versa

# INSTALLATION (WINDOWS)

- Download & Install JDK (Java Development Kit)
  - Go to official Java website: <https://www.oracle.com/java/technologies/downloads/>
  - Choose the latest version under Java SE (Standard Edition)
  - Download the Windows x64 Installer (.exe)
  - Run the downloaded .exe file to install JDK



# INSTALLATION (WINDOWS)

- Set Environment Variables
  - Search for Environment Variables in Windows search bar → click “Edit the system environment variables”
  - In System Properties window: Click “Environment Variables” → Under “System variables”, click “New”
    - Variable name: JAVA\_HOME
    - Variable value: Path to your JDK (e.g.: C:\Program Files\Java\jdk-23)
    - Find the variable ‘Path’ → click ‘Edit’ → add a new entry %JAVA\_HOME%\bin
- Verify Installation
  - Open Command Prompt (cmd) and type:

```
java -version
javac -version
// If both command show version numbers,
means that installation is successful
```

# INSTALLATION (MAC)

- Download & Install JDK (Java Development Kit)
  - Go to official Java website: <https://www.oracle.com/java/technologies/downloads/>
  - Choose the latest version under Java SE (Standard Edition)
  - Download the .dmg installer for macOS (use x64 for Intel processor Mac - older Macbooks and iMacs typically before 2020, use ARM64 for M1, M2, M3, or newer Apple Silicon chips {Can check by typing `uname -m` in Terminal})
  - Run the downloaded .dmg file to install JDK
- Verify Installation
  - Open Command Prompt (cmd) and type:

```
java -version
javac -version
// If both command show version numbers,
means that installation is successful
```

# INSTALLATION (LINUX)

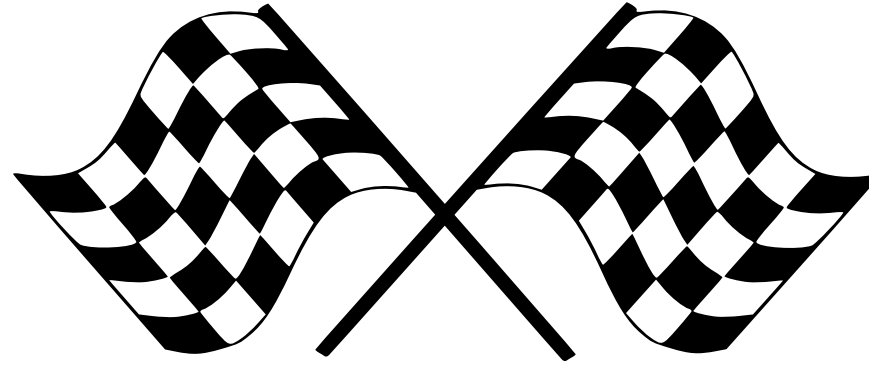
- Install using APT (Recommended)

```
sudo apt update  
sudo apt install openjdk-21-jdk -y
```

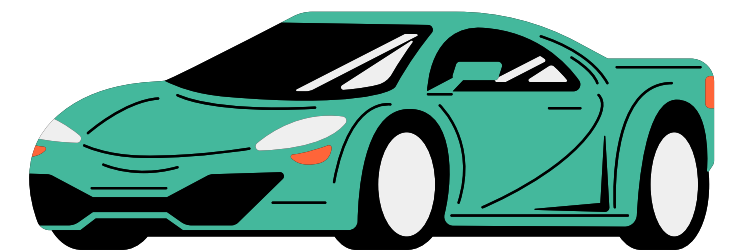
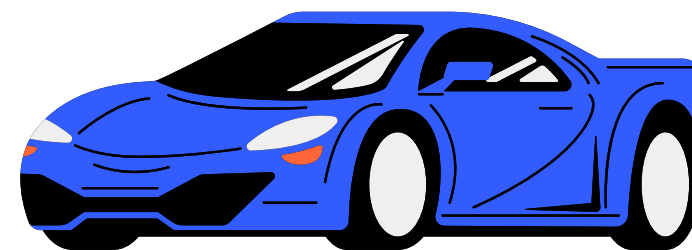
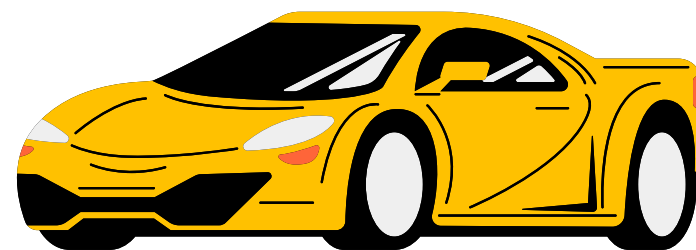
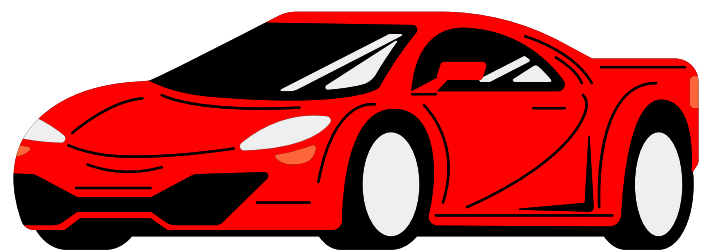
- Verify installation

```
java -version  
javac -version  
// If both command show version numbers, means that  
installation is successful
```





# BASIC STRUCTURE



# BASIC STRUCTURE OF A JAVA PROGRAM

- A typical Java program includes:
  - Class definition: Every code in Java must be inside a class. Class name should always start with an uppercase letter
  - Main method: Entry point of a Java program (`public static void main(String[] args)`)
- Case-sensitive: "MyClass" and "myclass" has different meaning
- Name of the java file must match the class name (class name of HelloWorld should have the filename of HelloWorld.java)

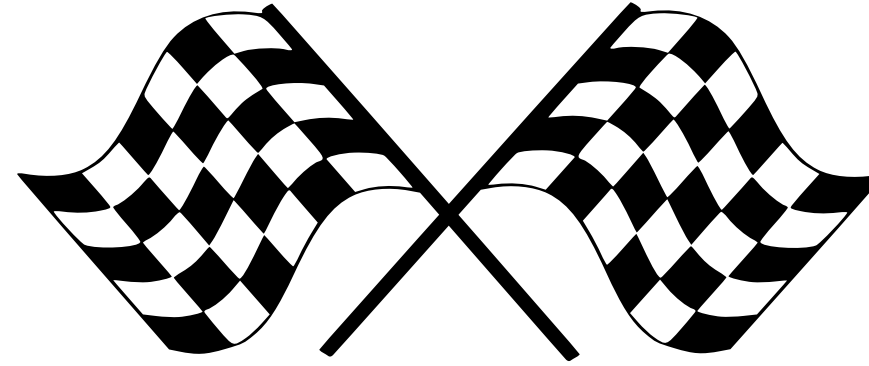
# BASIC STRUCTURE OF A JAVA PROGRAM

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

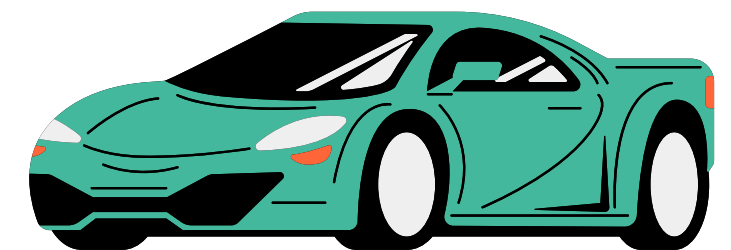
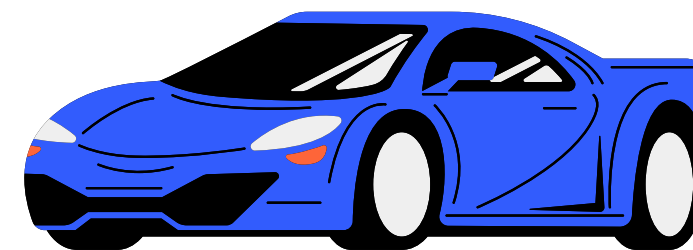
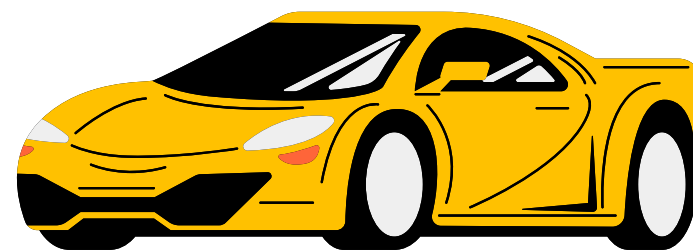
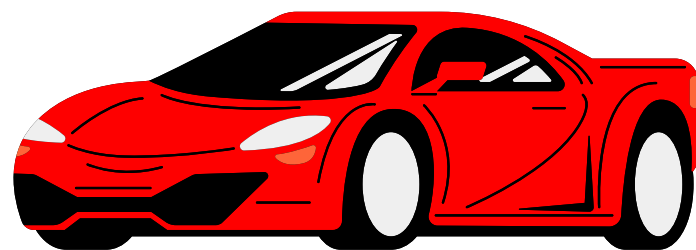
Define a class

Main method: Program entry point

Print output



# INPUT AND OUTPUT



# COUT, BUT JAVA

- cout -> System.out.print([OUTPUT])

```
System.out.print("I am learning Java.");
```

- cout with endl -> System.out.println([OUTPUT])

```
System.out.println("I am learning Java.");
```



# CIN, BUT JAVA

- Involves the use of `Scanner` class, found in `java.util` package

- Steps:

1. Import Scanner class from java.util package
2. Create object of the class
3. Use the available methods according to expected data type  
(mismatch will result in an error)

| Method      | Data Type |
|-------------|-----------|
| nextLong()  | long      |
| nextShort() | short     |
| nextByte()  | byte      |

| Method        | Data Type |
|---------------|-----------|
| nextBoolean() | boolean   |
| nextDouble()  | double    |
| nextFloat()   | float     |
| nextInt()     | integer   |
| nextLine()    | String    |

# CIN, BUT JAVA

```
import java.util.Scanner;
```

→ Import Scanner class from java.util package

```
class Main {  
    public static void main(String[] args) {
```

```
        Scanner myObj = new Scanner(System.in);
```

→ Create a Scanner object

```
        System.out.println("Enter name, age and salary:");
```

```
        String name = myObj.nextLine();
```

```
        int age = myObj.nextInt();
```

```
        double salary = myObj.nextDouble();
```

→ Read user input and store to variable

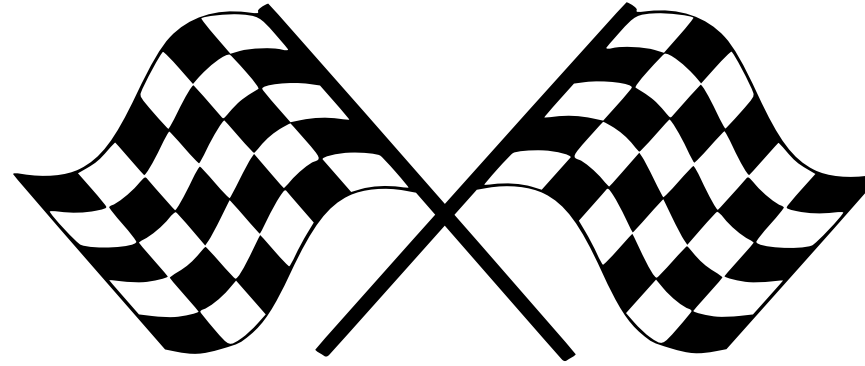
```
        System.out.println("Name: " + name);
```

```
        System.out.println("Age: " + age);
```

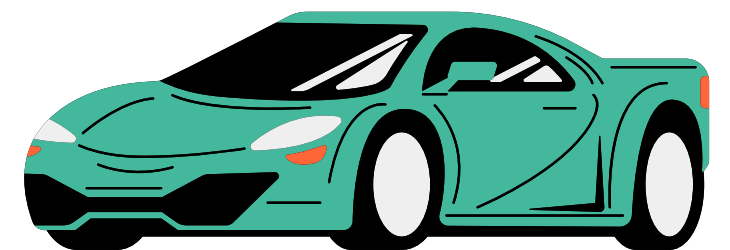
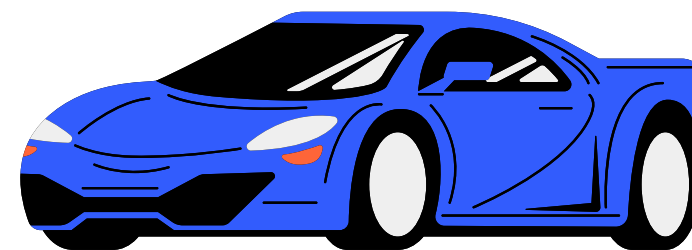
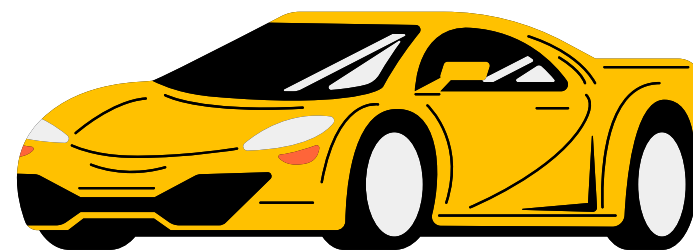
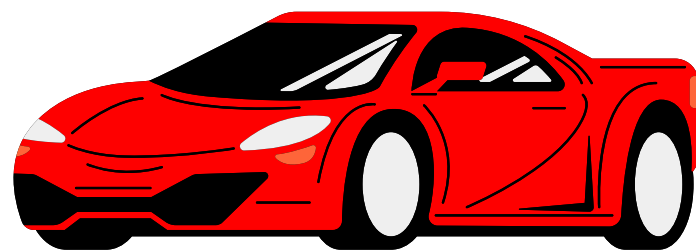
```
        System.out.println("Salary: " + salary);
```

→ Print the stored input

```
    }  
}
```



# DATA TYPES





# PRIMITIVE DATA TYPES

Specifies the size and type of variable values has no additional methods [1]

| Data Type | Size    | Description                                                                |
|-----------|---------|----------------------------------------------------------------------------|
| byte      | 1 byte  | Whole numbers from -128 to 127                                             |
| short     | 2 bytes | Whole numbers from -32,768 to 32,767                                       |
| int       | 4 bytes | Whole numbers from -2,147,483,648 to 2,147,483,647                         |
| long      | 8 bytes | Whole numbers from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807 |

# PRIMITIVE DATA TYPES

Specifies the size and type of variable values has no additional methods [1]

| Data Type | Size    | Description                                |
|-----------|---------|--------------------------------------------|
| float     | 4 bytes | Fractional numbers (between 6 to 7 digits) |
| double    | 8 bytes | Fractional numbers (15 digits)             |
| boolean   | 1 bit   | true or false values                       |
| char      | 2 bytes | a single character/letter or ASCII values  |

\*1 byte is 8 bits

# PRIMITIVE DATA TYPES

```
public class VariablesExample {  
    public static void main(String[] args) {  
        int num = 10;  
        double price = 19.99;  
        boolean isAvailable = true;  
        char grade = 'A';  
  
        System.out.println("Number: " + num);  
        System.out.println("Price: " + price);  
        System.out.println("Is Available: " + isAvailable);  
        System.out.println("Grade: " + grade);  
    }  
}
```

# NONPRIMITIVE DATA TYPES

Instances of objects have associated methods that can be called to perform operations

| Data Type | Description                                         |
|-----------|-----------------------------------------------------|
| String    | Represents a sequence of characters                 |
| Array     | Objects that store multiple values of the same type |
| Enums     | A fixed set of constants                            |

Other examples include Classes, Interfaces, Wrapper Classes, and Collections

# NONPRIMITIVE DATA TYPES - STRING

```
public class Main {  
    public static void main(String[] args) {  
  
        String greeting = "Hello, World!";  
        System.out.println("Original Greeting: " + greeting);  
        System.out.println("Greeting in Uppercase: " + greeting.toUpperCase());  
        System.out.println("Greeting Length: " + greeting.length());  
  
    }  
}
```



# NONPRIMITIVE DATA TYPES - ARRAY

```
public class Main {  
    public static void main(String[] args) {  
  
        String[] names = {"Alice", "Bob", "Charlie"};  
        System.out.println("\nNames in the array:");  
        for (String name : names) {  
            System.out.println(name);  
        }  
    }  
}
```

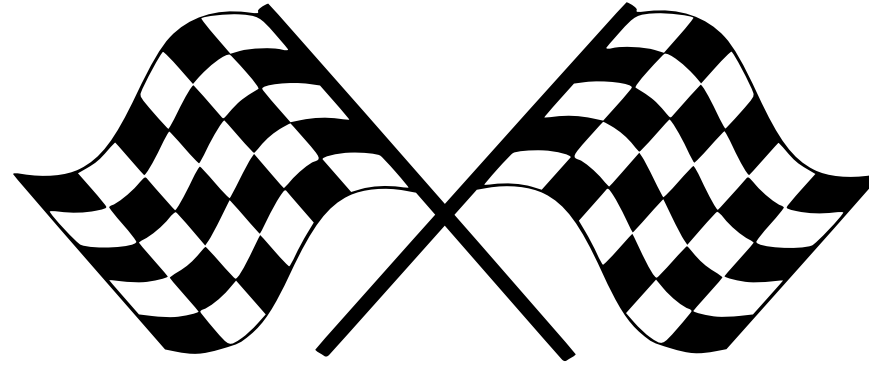
# NONPRIMITIVE DATA TYPES - ENUMS

```
enum Day { MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY }

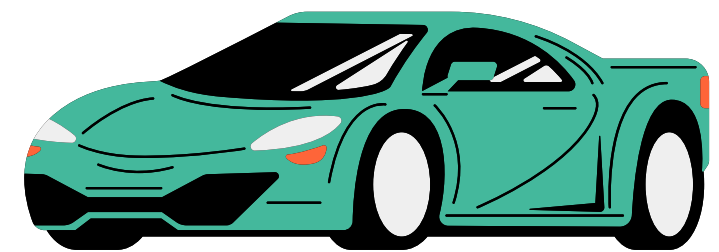
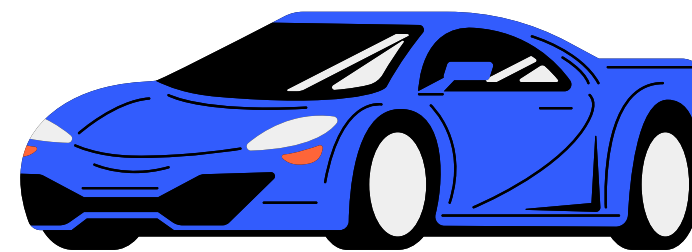
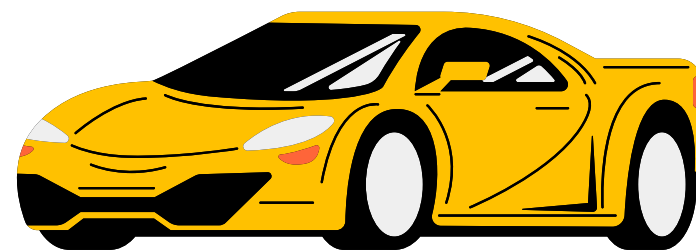
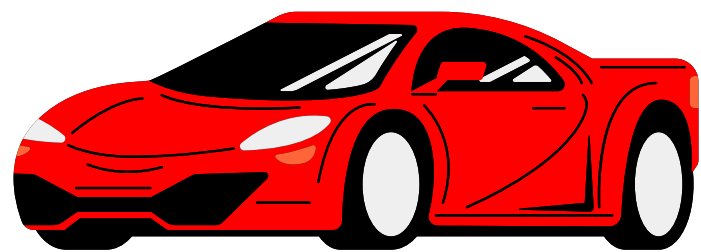
public class Main {
    public static void main(String[] args) {

        Day today = Day.WEDNESDAY;
        System.out.println("\nToday is: " + today);

        Day[] workdays = {Day.MONDAY, Day.TUESDAY, Day.WEDNESDAY, Day.THURSDAY, Day.FRIDAY};
        System.out.println("\nWorkdays are:");
        for (Day day : workdays) {
            System.out.println(day);
        }
    }
}
```



# CONDITIONALS AND LOOPS





# CONDITIONALS AND LOOPS

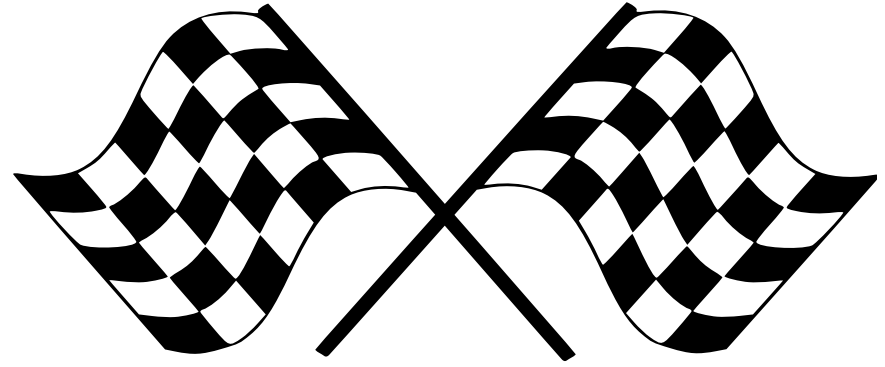
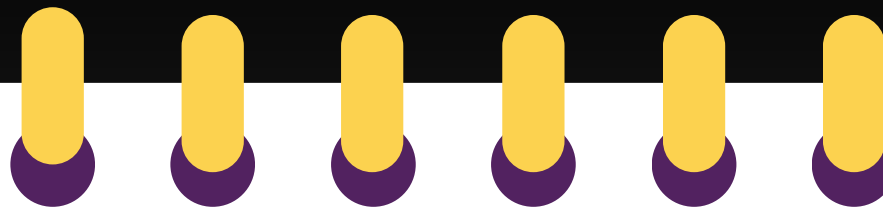
Similar to C++

Conditionals:

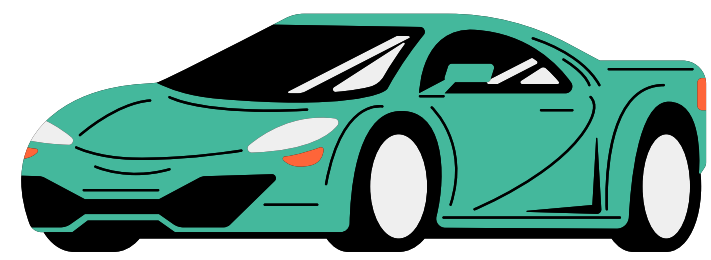
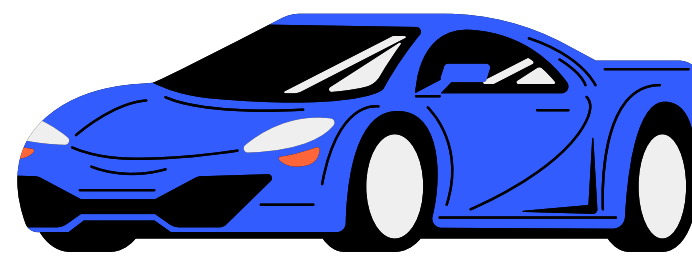
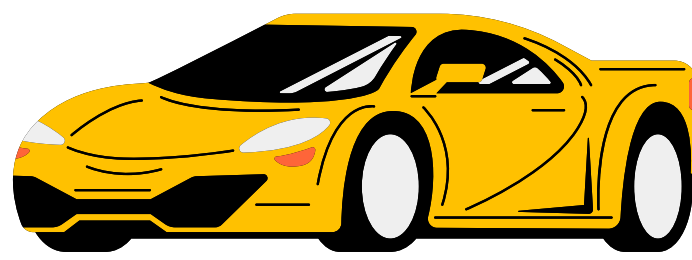
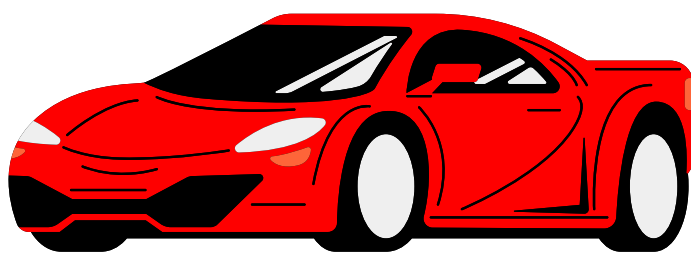
- if
- else if
- else

Loops:

- for
- while
- do-while



# METHOD / FUNCTIONS ?



# METHODS = FUNCTIONS ?

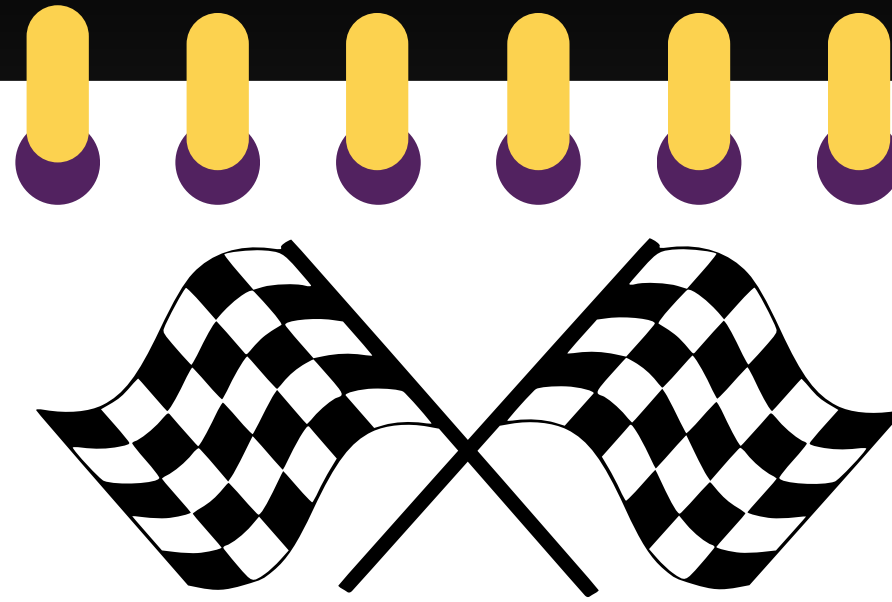
Method: A block of code that belongs to a class and performs a specific task

Function: A standalone block of code that performs a task and returns a result

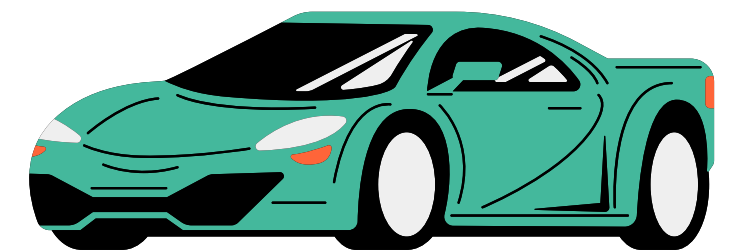
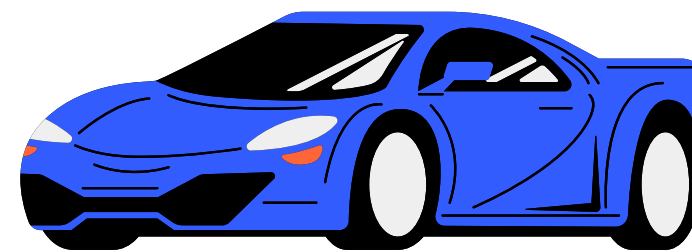
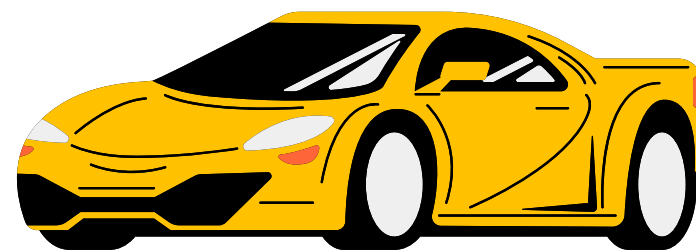
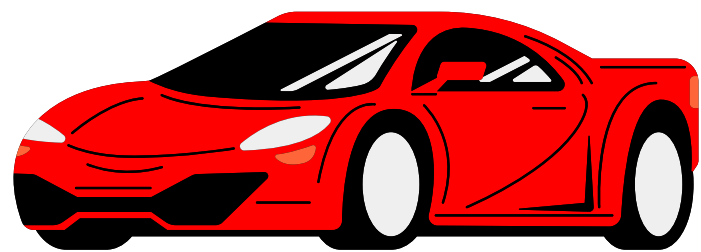
*So in Java we technically only have methods, because codes in Java need to be located inside classes*

# METHOD EXAMPLE

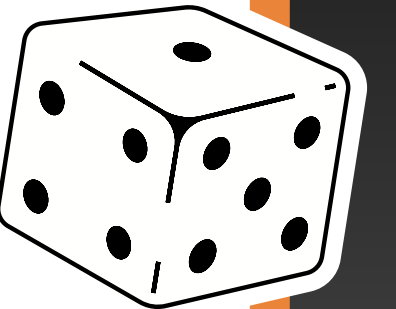
```
public class MethodExample {  
  
    public static int square(int number) {  
        return number * number;  
    }  
  
    public static void main(String[] args) {  
        int result = square(5);  
        System.out.println("Square of 5: " + result);  
    }  
}
```



ANY QUESTIONS?







THANK YOU FOR  
YOUR ATTENTION!

